

Instructions

AI Usage Policy:

It's okay to use AI tools to help with this programming assignment, but if you're using it to write code, make sure you understand what each line of that code is doing. Don't just copy and paste—read through it, test it, and be sure you can explain how it works. If I ask you about any part of the code you submitted, you should be able to walk me through it confidently. The goal is to learn, not just get the answer.

Checking:

You need to demo your working code to your instructor (either today or next week)

In this assignment, you'll write a client that will use sockets to communicate with a server that you will also write. Here's what your client and server should do:

1. Client:

Your client should first accept an integer between 1 and 100 from the keyboard, open a TCP socket to your server and send a message containing:

- a. a string containing your name (e.g., "Client of John Q. Smith") and
- b. the entered integer value and then wait for a server reply.

Your client should read the message sent by the server and display its name, the server's name, its integer value, and the server's integer value, and then compute and the sum. The client then terminates after releasing any created sockets. As an aside (and as a check that you are doing things correctly, you should make sure for yourself that the values and the sums are correct!)

2. Server:

Your server will create a string containing its name (e.g., "Server of John Q. Smith") and then begin accepting connections from clients. On receipt of a client message, your server should

1. print (display) the client's name (extracted from the received message) and the server's name
2. itself pick an integer between 1 and 100 (it's fine for the server to use the same number all the time) and display the client's number, its number, and the sum of those numbers
3. send its name string and the server-chosen integer value back to the client
4. if your server receives an integer value that is out of range, it should terminate after releasing any created sockets. You can use this to shut down your server.

Documentation:

Note that a short design document is required.

- You should program your client and server to print an informative statement whenever they take action (e.g., send or receive a message, detect termination of input, etc.) so that you can see whether your processes are working correctly (or not!). This also allows the instructor to determine from this output if your processes are working correctly.
- You should hand in screenshots (or file content, if your process is writing to a file) of these informative messages as well as the required output of the client and server (name strings, integer values, and sums)
- If you use an AI tool, you need to document the prompts you used, what part of the code is generated by the AI, and what you have written yourself. Moreover, you must add comments on the code on what parts of the code are doing.

Programming Languages and Operating Systems

You may write your programs in Java, C++, Python, or C on any platform (Linux/unix, Mac, PC) you want; you have accounts in the edlab. Other languages are OK, but ask me first. You can do this assignment on a linux/unix computer, a Mac or a PC. See

<http://www.lowtek.com/sockets/> for links to tutorials on the WINSOCK API, which we haven't covered in class.

Programming the assignment in C

If you choose to write your programs in C, you'll need to master the C system calls needed for socket programming. Personally, C is my favorite language for network programming since it gets you closest to the operating system's socket-related system calls. These include `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, and `close()`. For a quick text-only tutorial of socket programming specifically under Linux, see <http://www.lowtek.com/sockets/>. Here's another nice tutorial: <http://beej.us/guide/bgnet/>

Programming the assignment in Python.

A Python socket tutorial can be found in <http://docs.python.org/howto/sockets.html>

Programming notes

Here are a few tips/thoughts to help you with the assignment:

- You must choose a server port number greater than 1023 (to be safe, choose a server port number larger than 5000). If you want to explicitly choose your client-side port, also choose a number larger than 5000.
- You may need to know your machine's IP address, when one process connects to another. You can telnet to your own machine and seeing the dotted decimal address displayed by the telnet program. You can also use the UNIX `nslookup` command. On Windows, see the `ipconfig` utility. On a Mac, you can run the terminal program and use the `ifconfig` command (just type in `ifconfig` or `ifconfig | grep "inet "`).
- Many of you will be running the clients and senders on the same UNIX machine (e.g., by starting up the receiver and running it in the background, then starting up the relay in the background, and then starting up the sender. This is fine; since you're using sockets for communication these processes can run on the same machine or different machines without modification. Recall the use of the ampersand to start a process in the background. If you need to kill a process after you have started it, you can use the UNIX kill command. Use the UNIX `ps` command to find the process id of your server.
- Make sure you close every socket that you use in your program. If you abort your program, the socket may still hang around and the next time you try and bind a new socket to the port ID you previously used (but never closed), you may get an error. Also, please be aware that port ID's, when bound to sockets, are system-wide values and thus other students may be using the port number you are trying to use.

Extra Credit:

For extra credit, rewrite your server to be a concurrent server - that is a server that waits on the welcoming socket and then creates a new thread or process to handle the incoming request (i.e., carry out the username verification and message-of-the-day protocol described above). If you're doing the assignment in C, you'll need to learn about the `fork()` system call.

Here is an example that will get you going:

http://www.tutorialspoint.com/unix_sockets/socket_server_example.htm

If you're doing the assignment in Java, see the section "Supporting Multiple Clients" in Sun's socket programming tutorial,

<http://java.sun.com/docs/books/tutorial/networking/sockets/clientServer.html>.

Here's a page that will get you started that will get you started in Python:

<http://ilab.cs.byu.edu/python/threadingmodule.html> and

<http://www.eurion.net/python-snippets/snippet/Threaded%20Server.html>.

If you find better pages for C, Java or Python, please share with the class!

Optional: (But this will help you in your MP later)

You've got the basic assignment done. The last part of the assignment should be a fun, social thing to do. With your groupmates, get your client to interact with their server or vice versa. Note that you should be in the same network to be able to do this (I know this may be hard with the hybrid setup since this may need to be done in person. That's why this is optional). If you've got a server up and running, you can advertise your server's services to anyone in the class (you're not allowed to charge for service!), and if you've got a client, all you need to do is interact with such an advertised server.

Note that if you've got your own client and server running, there isn't any more programming involved – just running your client and server with someone else's. This shouldn't be very hard at all! For this part of the assignment, you need only hand in the output from your “wide” – the client or the server, since you'll be printing out the name of the other student's client or server with whom your client or server is interacting. Recall that when we discussed RFC standards for protocols we noted that the IETF requires that two independent implementations of a protocol must interoperate; that's what you are doing here. Again, make sure to check that your numbers and sums are correct.

Screenshots

Server Operations:

1. Client not connected

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\valos\OneDrive\Desktop\2nd Year Term 3\CSNETWK>python server.py
SERVER: Listening on Valos:5555

SERVER: Waiting for connection...
|
```

2. Client connected & finished input

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\valos\OneDrive\Desktop\2nd Year Term 3\CSNETWK>python server.py
SERVER: Listening on Valos:5555

SERVER: Waiting for connection...
SERVER: Connection from ('192.168.88.3', 51387)
Client Name: Client of John
Server Name: Server of Alice
Client Number: 100
Server Number: 50
Sum: 150
SERVER: Reply sent.
SERVER: Client disconnected.

SERVER: Waiting for connection...
|
```

Client Operations:

1. Valid Number Input

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\valos\OneDrive\Desktop\2nd Year Term 3\CSNETWK>python client.py
Enter a number (1-100): 100
CLIENT: Connecting...
CLIENT: Connected.
CLIENT: Message sent.

CLIENT: Reply received.
Client Name: Client of John
Server Name: Server of Alice
Client Number: 100
Server Number: 50
Sum: 150
CLIENT: Socket closed.
```

2. Invalid Number Input

```
C:\Users\valos\OneDrive\Desktop\2nd Year Term 3\CSNETWK>python client.py
Enter a number (1-100): 1029109334923
Number must be between 1 and 100.
Enter a number (1-100): |
```

3. NaN input

```
C:\Users\valos\OneDrive\Desktop\2nd Year Term 3\CSNETWK>python client.py
Enter a number (1-100): dfsdfkjdsfkh
Invalid input. Enter an integer.
Enter a number (1-100): |
```

Required Outputs

Required Outputs

INPUT	OUTPUT	
	Client	Server
int num (1-100)	Connecting... Connected. Message sent. Once received: Reply received. Client Name: String Server Name: String Client Number: int Server Number: int Sum: int Client: Socket closed.	
int num <1 or >100	Number must be between 1 and 100.	
NaN	Invalid input. Enter an integer.	
		SERVER: Waiting for connection... SERVER: Connection from ('HOST', PORT) Client Name: String Server Name: String Client Number: int Server Number: int Sum: int After sending client reply: SERVER: Reply sent. After client disconnects: SERVER: Client disconnected.